

Learning ~~Intent-Aligned~~ ~~Task-Relevant~~ Latent Residuals for ~~Language-Conditioned~~ Robot Manipulation

Anonymous submission

Abstract— Learning ~~language-conditioned~~ ~~language-guided~~ robot manipulation policies requires representations that capture not only the current scene but also how it changes under task instructions. ~~While Existing~~ general-purpose visual representations ~~have improved robotic perception, they are mainly focused on encoding static scenes and typically encode the current visual scene and hence provide limited guidance on the temporal dynamics relevant to robotic actions.~~ We propose **LARM**, a self-supervised ~~learning pre-training~~ framework that extracts task-relevant ~~temporal dynamics~~ ~~future changes from action-free human videos~~ and represents them as latent residuals. ~~These intent-aware residuals provide a clear and direct target for the robot actions, for downstream policy learning.~~ During pre-training, a shared future predictor operates with asymmetric visual access: both use the global visual context and instruction, while the full-information stream has additional access to dense visual tokens. The other stream, called intent stream, is guided by the full-information prediction to capture task-relevant future changes. During policy learning, **LARM** bridges the gap between pre-training and deployment by retaining the predictive module to compute an explicit latent residual between the intent stream predicts the future latent state from the current observation and the predicted future state. This lightweight, control-oriented signal highlights the necessary actions directly, which simplifies the learning process of the policy. Experiments in both simulated and instruction, and we compute its residual from the current latent state. The policy is conditioned on both the current representation and this residual, providing an explicit task-relevant change signal for action prediction. Experiments on two simulation benchmarks and seven real-robot ~~manipulation environments~~ tasks demonstrate that these latent residuals serve as an effective prior for policy learning, ~~outperforming prior methods.~~

I. INTRODUCTION

Robot learning is increasingly important to build autonomous systems that operate in unstructured and unseen environments. Learning robot manipulation policies fundamentally requires rich, structured representations that capture object semantics and geometry, spatial relationships, dynamics, and task-relevant affordances. However, learning such representations without extensive manual supervision remains a fundamental challenge.

Overview. (Top) Existing models discard predictive knowledge after pre-training, lacking guidance on the temporal evolution of task-relevant dynamics. (Bottom) Our approach utilizes the *intent-aware latent residual* (Δz) as a guidance signal that simplifies complex manipulation into a precise residual-tracking task, effectively bridging the gap between high-level semantics and low-level control.

Recent advances in self-supervised Learning-based robot manipulation requires visual representations that describe not

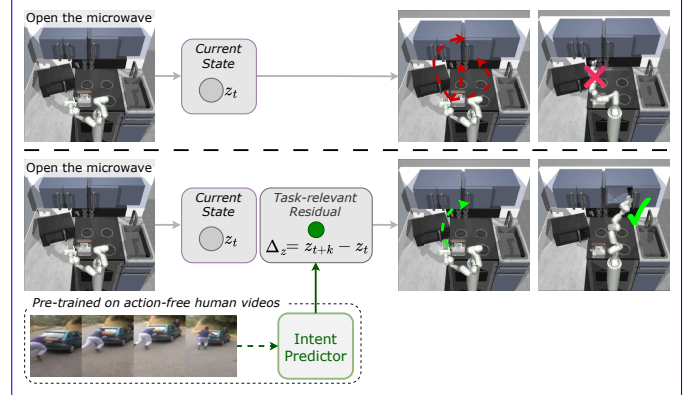


Fig. 1: **Overview.** (Top) Existing approaches provide the policy with the current visual state, leaving the downstream controller to infer task-relevant changes. (Bottom) Our method uses the intent stream that provides task-relevant residual Δz as context for the closed-loop policy.

only what is currently visible, but also how the scene should change for a task to succeed. While recent advances in visual representation learning have enabled general-purpose models, such as DINO [1], R3M [2], and VIP [3], which provide transferrable features for frozen visual features to support robotic perception and manipulation. These models improve robots’ ability to extract semantically meaningful information from the scene, but manipulation is inherently temporal, as effective control requires a policy to identify which object motion, contact event, or configuration change matters next. However, these representations are fundamentally static, as they are learned to characterize the current state of the scene (*what is in the scene*), rather than capturing temporal transition (*how the world is changed*). In result, downstream control policies are required to learn the capability of inferring many generic representations either lack temporal knowledge or encode it only implicitly. Static features do not directly indicate how the scene is expected to evolve, forcing downstream controllers to infer task-relevant dynamics and physical transitions, implicitly discovering which visual changes are critical to task completion. This separation between perception and dynamics understanding forces policies to re-learn, from scratch, the temporal structure that is essential for manipulation from limited demonstrations [4], [5].

Learning predictive representations offers a natural solution by introducing a temporal structure through future-state forecasting. By learning to predict the future state, a model can capture the dynamics that policies

Predictive representation learning provides a natural way to model these temporal changes, while existing predictive pipelines do not directly employ a control-oriented change signal to the policy. Future prediction has long been used to learn dynamics useful for decision making [6], [7]. However, existing approaches show two critical limitations. First, many approaches rely on computationally heavy pixel-level reconstructions [8]–[13], forcing models to allocate a capacity to learn high-frequency. In vision-based representation learning, many methods train by reconstructing or predicting future visual content [8]–[13]. Such objectives can allocate capacity to visual details that might be irrelevant to operating robotic tasks. This entangles manipulation-induced object motion with passive dynamics or environmental noise. Second, existing models typically discard the predictive decoder entirely during downstream policy training [2], [14], [15]. By retaining only the static encoder (Fig. 1, top), these models discard dynamics information that predictive learning is designed to capture, forcing downstream task policies to re-solve the disentanglement problem from scratch.

In this work, we hypothesize that predictive learning should not be treated only as a means to extract information for manipulation. Moreover, the temporal component is often discarded or compressed after pre-training task that will be discarded, but rather as a mechanism for extracting *intent-aligned state transitions* directly within the latent space. Instead of predicting all future environmental changes, we focus on identifying specific future states that align with the intended task: predictive decoders may be removed before downstream learning [14], [15], while objectives such as time-contrastive learning [2] leave temporal information only implicitly inside a static embedding. The challenge is therefore not simply how to learn future prediction, but how to retain the task-relevant part of prediction as an explicit policy input.

We present LARM (Learning – Intent-Aligned Latent Residuals for Language-Conditioned Robotic Manipulation) propose TRAIL (Task-Relevant residuals as an Action prior for robot manipulation), a self-supervised representation learning framework that constrains future latent representations using task semantics derived from an image and language instructions. Specifically, LARM uses two distinct prediction modules. One module has access to detailed visual features of the scene, while framework that learns a task-relevant latent change from action-free human videos¹. TRAIL defines current and future states in a frozen visual feature space and trains a shared future predictor with asymmetric visual access. The full-information stream uses the other module relies on a global scene-level representation. By encouraging these two modules to align their predicted future states, the model learns to filter out irrelevant background or passive movements and focuses on the visual changes that are directly related to the robot

executing the target manipulation task.

global visual token, dense visual patch tokens, and grounded language context, while the intent stream uses only the global visual token and language context. Distilling the full-information prediction into the intent stream encourages the model to capture task-relevant future changes rather than generic background motion or passive scene changes. At deployment, we use LARM’s predictive module to compute a lightweight latent residual between the current observation and the future state (Fig. 1, bottom). This residual directly captures intent-aligned changes specified by the language-based task description. This residual provides an explicit directional signal, which simplifies the policy’s objective to follow a clear, step-by-step latent trajectory rather than inferring task priorities from complex observations. Consequently, the predictor actively guides downstream control rather than merely serving as a standalone pre-training objective. The intent stream to predict the task-relevant future latent and compute its residual from the current state. We design the policy conditioned on both the current representation and this residual, providing an explicit task-relevant change signal for downstream control.

We summarize our contributions. Our contributions are summarized as follows:

- We introduce an asymmetric predictive representation learning framework that extracts intent-aligned dynamics in latent space. Our latent-space learning naturally encourages the model to focus on task-relevant changes without requiring explicit supervision. captures task-relevant latent dynamics from action-free human videos.
- We show that utilizing the predictor is beneficial, by demonstrating that the intent-aware latent residuals (Δ_z) can serve as a lightweight, effective control prior that improves policy precision and robustness.
- We validate our model on the simulated Franka Kitchen benchmark propose to retain the trained predictor and explicitly feed the task-relevant latent change signal to downstream policies as part of their context.

Experiments on two simulated benchmarks, Franka Kitchen [16] and across 7 different Meta-World MT10 [17], and seven real-robot manipulation tasks, where our model consistently outperforms existing models, demonstrate consistent improvements over existing visual and predictive representations.

II. RELATED WORK

A. Visual representations for robotic manipulation

Vision foundation models have become widely used for robotic representation learning. Self-supervised approaches extract robust features from static observations via time-contrastive learning (e.g., R3M [2], VIP [3], LIV [18]) and masked autoencoders (e.g., MAE [8], MVP [14], VC-1 [19]). To incorporate semantic priors, vision-language models align visual features with language instructions, enabling open-vocabulary manipulation (e.g., CLIPort [20], RT-1 [21]).

¹For more results: <https://anonymous.4open.science/w/ICRA2027-E3D6/index.html>

These methods ~~primarily~~ produce static representations that capture *what* is in the scene, but not *how* it changes over time. To extract more robust and task-relevant signals, recent approaches extend these static representations using the Information Bottleneck (IB) principle [22]. By strictly limiting information capacity, IB-based methods learn compressed representations that filter out irrelevant distractors, improving downstream reinforcement learning [23] and behavior cloning [24]. However, these IB-based methods still lack explicit modeling of ~~task-conditioned~~ task-relevant dynamics, as they compress the current state while not predicting its future evolution.

B. Predictive modeling for visual control

An alternative approach is learning representations by predicting future observations, which effectively captures environment dynamics. Several approaches focus on pixel-level video prediction (e.g., RoboNet [25], UniPi [26], SUSIE [27]), for visual planning or sub-goal generation. Other approaches, such as Voltron [12] and LaVA-Man [10], extend such pixel-level prediction by integrating language and action conditioning into image reconstruction and forward video prediction. Latent action models [28]–[30] learn the action conditioning without action labels. These models infer a low-capacity latent action from the current and future frames and use it to reconstruct the future frame’s pixels from the current frame. The latent actions are then used to pre-train policies, but must later be mapped to executable actions [30], [31]. However, forecasting at the pixel level is computationally expensive and overly sensitive to task-irrelevant visual distractions such as lighting variations or background movement [32], [33].

Joint-Embedding Predictive Architectures (JEPAs) [32] address the limitation of pixel-level prediction by shifting the prediction objective to the latent space, which improves efficiency. I-JEPA [34] and V-JEPA [33] learn powerful spatial and spatiotemporal representations by predicting the latent embeddings of masked regions. In the robotics domain, MC-JEPA [35] jointly models motion and content features, which can capture both scene appearance and dynamics. As a recent extension to predictive models, ToBo (Token Bottleneck) [13] squeezes the current scene into a compact bottleneck token to predict future latent patches, forcing the model to encode temporal dependencies.

Despite these advances, standard latent predictive models (e.g., Siamese MAE [9]) still attempt to forecast the *full* future state, inevitably entangling controllable transitions with uncontrollable passive dynamics. In contrast, our proposed LARM-TRAIL combines the efficiency of latent prediction with an intent-conditioned a task-relevant bottleneck. Unlike prior methods, LARM explicitly predicts we explicitly produce the *residual* between current and future states, providing policies with a gradient-like directional cue that isolates task-compatible dynamics task-relevant dynamics without requiring any mapping to executable actions.

LARM architecture. During pre-training, visual and linguistic features are fused using the bidirectional

cross-attention module. An asymmetric architecture suppresses the futures that cannot be predicted from task context between the intent-aligned predictor (top) and the full-feature predictor (bottom) to filter out task-irrelevant dynamics. During deployment, the full predictor is discarded. The future latent output by the full predictor, denoted as z_{t+1} , is then subtracted by the current latent z_t to obtain the residual Δ_z which explicitly guides downstream policy learning.

III. METHOD

Our method leverages large-scale, general-domain human video datasets (e.g., web videos) for self-supervised pre-training rather than relying on robot-specific trajectories. This allows the model to learn generalized motion priors by understanding the correlation between visual dynamics and language-based action annotations in datasets like Kinetics [36] (e.g., “a person is water sliding”). By capturing the underlying structure of these motions, the model develops a robust understanding of physical dynamics, which can then be transferred to robotic control with minimal post-training. To this end, we formulate a two-stage framework: (1) self-supervised pre-training with intent-aligned prediction, which learns to isolate TRAIL learns task-relevant dynamics by predicting future latent states under an information bottleneck latent changes from action-free human videos and uses it as policy context. During pre-training, a language-grounded encoder and an asymmetric predictor learn to forecast future visual representations under different levels of visual access (Sec. ?? and ??), and (2) residual-guided policy learning, which conditions the downstream policy on the predicted change between current and future states III-A). During downstream imitation learning, the intent stream produces a latent residual that conditions the policy on task-relevant change (Sec. III-B).

A. Language-grounded visual encoding Language-guided asymmetric prediction

In Fig. 2, we show the model architecture for our self-supervised shows language-guided asymmetric pre-training. The pre-training begins by grounding visual features in natural language descriptions of general human activities. Inspired by GLIP [37] and recent vision-language fusion techniques for robotics [12], we employ bidirectional cross-attention to fuse information from the two modalities. This fusion process enhances a semantic consistency between task intent and scene features, allowing the model to compress task-relevant information into the global context representation, i.e., the CLS token Given a current frame I_t , a future frame I_{t+k} at offset k , and a language instruction l , TRAIL learns to predict the future latent target z_{t+k}^{gt} from the current latent z_t and l . To disentangle task-relevant future changes from general visual dynamics, TRAIL predicts the same future target with different levels of visual access. In particular, the full stream has access to both global and dense visual information, while the intent stream is

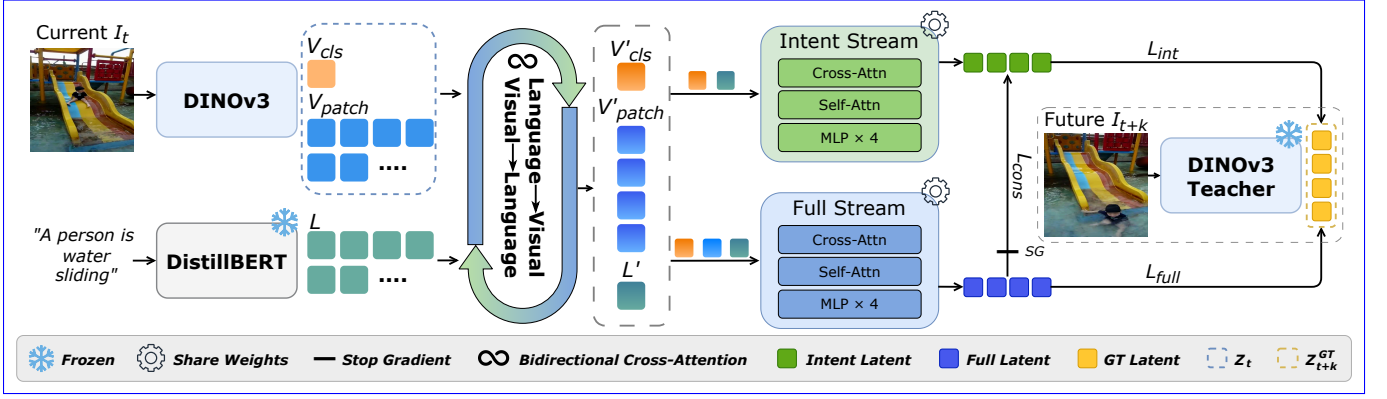


Fig. 2: **TRAIL architecture in pre-training.** Visual and language tokens are first fused by bidirectional cross-attention. The same predictor is then evaluated under two memory-access conditions: the full-information stream reads class, patch, and language tokens, whereas the intent stream reads only the class and language tokens. Both streams predict the frozen future target, and the full-information prediction provides a stopped consistency target for the intent prediction. The modules with snowflakes mark are frozen, and all other modules are set to trainable. At deployment, only the intent stream is retained to produce the predicted future latent $\hat{z}_{t+k}^{\text{int}}$ for the downstream policy.

restricted to global visual and language context. Both streams are supervised by the same future latent target, with the full prediction further guiding the intent prediction through consistency learning.

Given-visual We first use a trainable visual encoder [38] to map I_t to the visual-token sequence z_t , while a frozen copy of the encoder maps I_{t+k} to the supervision target z_{t+k}^{gt} . At an input resolution of 224×224 , each visual sequence contains one class token V_{cls} and 196 patch tokens V and language tokens L , we first apply intra-modal self-attention to build rich internal representations for each modality independently. We then fuse two modalities through two cross-attention layers for bi-directional feature updates:

$$\begin{aligned} V' &= \text{CrossAttn}(V, L, L) + V, \\ L' &= \text{CrossAttn}(L, V', V') + L, \end{aligned}$$

where the first, second, third arguments for the cross attention layers, $\text{CrossAttn}(\cdot, \cdot, \cdot)$, are queries, keys, and values, respectively. As shown in Eq. ?? and Eq. ??, these cross-attention layers have two effects: visual patches can be conditioned on the language context (attention shifts to regions mentioned in the instruction), while language tokens can be contextualized by the current scene layout (the same instruction yields different grounding depending on object locations). The resulting CLS token thus serves as a semantic anchor that encodes both visual content and task intent. Meanwhile, the sequence of updated patch embeddings preserves dense, localized spatial features, serving as the foundational input for fine-grained spatial prediction in the subsequent stage V_{patch} . A frozen DistilBERT encoder [39] maps the instruction l to the language-token sequence L . The visual tokens, V_{cls} and V_{patch} , and L are then fused through bidirectional cross-attention. The visual tokens first query L with a residual connection, producing the instruction-conditioned class token V'_{cls} and patch tokens

V'_{patch} . The language tokens subsequently query these updated visual tokens, producing the scene-conditioned language tokens L' . The class token provides a compact task-conditioned representation, while the patch tokens preserve dense spatial information.

B. Asymmetric intent-aligned prediction

We aim to ensure that the learned representations capture dynamics relevant to the task rather than random environmental variations. We introduce an intent-aligned predictive learning objective that filters out task-irrelevant background dynamics and forces the model to focus only on the core motions dictated by the language instruction. Existing predictive models forecast future observations based on past inputs, which may inevitably entangle controllable transitions with uncontrollable passive dynamics [8], [9]. We isolate task-relevant change without pixel-level decoding by employing an asymmetric predictive architecture in the latent space, as shown in Fig. 2.

Let z_{t+1}^{gt} be the ground-truth future latent state extracted by a frozen DINOv3 [38] target encoder which we defined as a teacher backbone in Fig. 2. During pre-training, the Teacher encodes the future frame to generate the ground-truth latent target providing supervision signal to our trainable module. We construct two parallel predictive streams with shared weights. First, the Full-Information Predictor (F_{full}) retains access to dense spatial memory (V'), enabling accurate modeling of physical scene evolution. Second, the Intent-aligned Predictor (F_{intent}) operates under an information bottleneck, restricted to only the

With the language-grounded CLS token. It lacks low-level spatial memory and relies on global task context (tokens V'_{cls} , V'_{patch} , and L') to infer future dynamics. we then perform asymmetric future prediction. The intent stream predicts the full future token sequence z_{t+k}^{gt} from only the instruction-conditioned class token V'_{cls} and language tokens L' , without direct access to current patch tokens

V'_{patch} . Because this restriction removes much current spatial information, we introduce a full-information stream during pre-training that also receives V'_{patch} . Its prediction, conditioned on the complete current visual state, serves as a consistency target for the intent stream. The two streams share the predictor P and future queries Q , differing only in their memory inputs:

$$\hat{z}_{t+k}^{\text{full}} = P(Q \mid [V'_{\text{cls}}, V'_{\text{patch}}, L']), \quad (1)$$

$$\hat{z}_{t+k}^{\text{int}} = P(Q \mid [V'_{\text{cls}}, L']). \quad (2)$$

Both predictions are supervised by z_{t+k}^{gt} . In addition, the stopped full-information prediction regularizes the intent prediction, encouraging the compact intent representation to preserve future-relevant information that is consistent with the detailed current scene. The full-information stream is used only during pre-training, and downstream policy learning retains only the intent stream.

Both predictors are optimized jointly to reconstruct future latent states. We use a dual-objective loss combining Mean-Squared-Error (MSE) and Cosine distance. Finally, we optimize both streams with full-prediction, intent-prediction, and consistency losses:

$$\mathcal{L}_{\text{full}} = \|F_{\text{full}}(V', L') - z_{t+k}^{\text{gt}}\|^2 + (1 - \cos(F_{\text{full}}, z_{t+k}^{\text{gt}})),$$

$$\mathcal{L}_{\text{intent}} = \|F_{\text{intent}}(V'_{\text{cls}}, L') - z_{t+k}^{\text{gt}}\|^2 + (1 - \cos(F_{\text{intent}}, z_{t+k}^{\text{gt}})),$$

$$\mathcal{L}_{\text{full}} = d(\hat{z}_{t+k}^{\text{full}}, z_{t+k}^{\text{gt}}), \quad (3)$$

The cosine distance function in Eq.

$$\mathcal{L}_{\text{int}} = d(\hat{z}_{t+k}^{\text{int}}, z_{t+k}^{\text{gt}}), \quad (4)$$

$$\mathcal{L}_{\text{cons}} = \text{MSE}(\hat{z}_{t+k}^{\text{int}}, \text{sg}(\hat{z}_{t+k}^{\text{full}})), \quad (5)$$

where $d(x, y) = \text{MSE}(x, y) + 1 - \cos(x, y)$ is the sum of mean-squared error and cosine distance between two aligned token sequences x and y . In Eqs. 3 and Eq. ?? provides scale-invariant supervision, aligning the semantic direction of predictions with the target, while MSE acts as a magnitude regularizer to keep predictions on the DINOv3 feature manifold.

Proposed residual-guided policy learning. The LARM encoder compresses the raw image into latent representations, where the future latent z_{t+1} is predicted from the current state z_t . The latent residual Δ_z captures the intended motion, which is fused with the current state, then forms the final input signal along with joint state for policy model.

Following asymmetric siamese designs [9], we introduce a consistency loss $\mathcal{L}_{\text{cons}}$ to penalize the divergence between the two predictive streams:

$$\mathcal{L}_{\text{cons}} = \alpha \cdot \|F_{\text{intent}} - \text{sg}(F_{\text{full}})\|^2,$$

4. $\mathcal{L}_{\text{full}}$ and $\mathcal{L}_{\text{int}}$ measure the prediction errors of the full-information and intent streams with respect to z_{t+k}^{gt} . In Eq. 5, $\mathcal{L}_{\text{cons}}$ encourages the intent prediction to agree

with the patch-informed full-information prediction, while $\text{sg}(\cdot)$ stops gradients through the full-information output. The overall pre-training loss becomes

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{full}} + \mathcal{L}_{\text{int}} + \alpha \mathcal{L}_{\text{cons}}, \quad (6)$$

where $\text{sg}(\cdot)$ denotes a stop-gradient operator, which prevents gradients from propagating through F_{full} , and $\alpha = 0.3$ is a hyperparameter to scale the effect of the loss. This encourages the intent-aligned stream to agree with α weights the consistency loss, empirically set to $\alpha = 0.3$. Since $\text{sg}(\cdot)$ is applied to the full-information stream while maintaining independence. Since F_{intent} operates under an information bottleneck, minimizing this consistency loss effectively filters F_{full} 's predictions: futures that cannot be predicted from task context alone are suppressed. The model thus learns to allocate capacity primarily to intent-aligned dynamics. The total objective combines Eq. 3, Eq. ??, and Eq. ??:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{full}} + \mathcal{L}_{\text{intent}} + \mathcal{L}_{\text{cons}}.$$

prediction, gradients from $\mathcal{L}_{\text{cons}}$ propagate only through the intent branch. The future-frame visual encoder and language encoder are frozen, while the current-frame visual encoder, language projection, fusion module, future queries, and asymmetric predictor are trainable.

B. Residual-guided policy learning

Following After pre-training, we adapt the learned representations for downstream manipulation. Existing approaches [10], [27] condition policies on fully reconstructed future images, which introduces significant computational overhead and generates redundant static information, e.g. unchanged static background. Therefore, we discard the full predictor and retain only F_{intent} , now denoted as z_{t+1} , to compute a latent residual: keep only the intent stream and use its predicted future latent to construct a task-relevant residual for downstream policy learning. Given the current observation and instruction, the intent stream produces $\hat{z}_{t+k}^{\text{int}}$ in the same frozen visual feature space as z_t . We then form the token-wise residual

$$\Delta_z = \hat{z}_{t+1} - \hat{z}_{t+k}^{\text{int}} - z_t, \quad (7)$$

The residual where Δ_z in Eq. 7 represents the minimal update between current and future states. In other words, Δ_z can be considered as a directional cue that highlights what should change to achieve the task. Rather than providing the policy with a full future state, which forces the policy network to implicitly compute the represents the task-relevant change from the current latent state. Unlike an absolute future feature, this representation makes the difference between current and future observations to deduce the required motion predicted states explicit rather than leaving the policy to infer that difference implicitly.

To translate this latent residual into motor commands, we condition the policy on both the current state z_t and the predicted change Δ_z . As illustrated in Fig. 3, As Δ_z is first fused with the current state z_t via

SNR-enhanced normalization. This operation amplifies the Δ_z signal, ensuring that it is not disregarded as background noise by the policy model, thus producing a combined representation $z_{fused} = \text{Norm}(z_t \oplus \Delta_z)$. This representation is then concatenated with proprioceptive joint states q_t and fed into the policy network $\pi(a_t | z_{fused}, q_t)$, which outputs actions via standard behavioral cloning or reinforcement learning objectives. Since the current-state features have a smaller magnitude than the current-state features, we normalize the two token sequences independently before passing them to the downstream policy. This prevents their scale difference from suppressing the residual during feature fusion. We then train the downstream policy by imitation learning using action-labeled robot demonstrations. During downstream policy learning, the pre-trained visual encoder, language-fusion module, and intent predictor are kept frozen, while only the action decoder is trained.

The latent residual Δ_z can be adapted to different policy architectures depending on the action space. For 1D continuous control that uses state-based algorithms predicting joint angles or end-effector poses [40], we flatten the CLS residual into a 1D vector and concatenate it with q_t . TRAIL can be integrated with different downstream policy architectures through the same residual representation. For simulation benchmarks, e.g. Franka Kitchen and Meta-World, we concatenate z_t and Δ_z with proprioception use them as input to a multi-layer perceptron (MLP) policy. In 2D spatial action policies designed for vision-based affordance models (e.g., tabletop an action decoder that outputs continuous control commands. For real-robot tabletop tasks, we reshape the patch tokens from both z_t and Δ_z into spatial feature maps and fuse them to predict language-conditioned affordance maps [20]. Thus, the same pre-trained TRAIL representations can be used across different action spaces by adapting only the downstream readout.

At deployment, the intent stream is executed at every control step in closed loop. Given the current image and instruction, the model predicts $\hat{z}_{t+k}^{\text{int}}$ and computes Δ_z , and then predicts an action. This process is repeated after each robot action, and the predictor is not rolled out over multiple future steps. Here a control step denotes one policy decision, whose granularity follows the action space. For the continuous-control benchmarks, one decision is a single low-level command, so the predictor is re-evaluated at the environment control frequency. For the tabletop affordance read-out, one decision is a complete pick-and-place [20], we extract the residual of patch tokens, reshape it into a 2D spatial feature map, and fuse it with the policy’s visual backbone via convolutional layers, providing an explicit spatial prior on where and how to interact. This flexibility allows LARM to integrate seamlessly with both low-dimensional state policies and high-dimensional visuomotor architectures without architectural modification. primitive: a single prediction supplies both endpoints of the motion from one observation, and the primitive is executed without re-observation before the next decision is made. Note

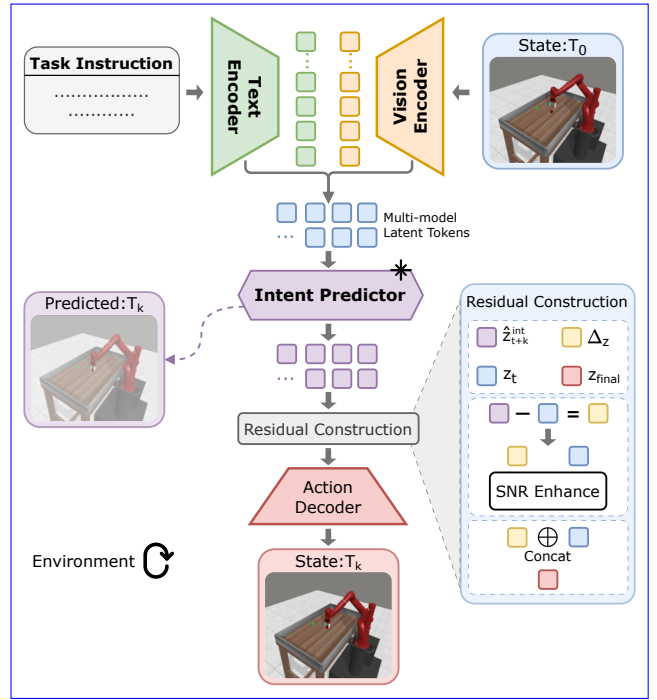


Fig. 3: **Proposed residual-guided policy learning.** The retained intent stream estimates a task-relevant latent forecast from the current observation and instruction. TRAIL subtracts the current latent state to obtain Δ_z , balances the state and residual feature scales, and then fuses them with proprioception for imitation learning.

that, during pre-training, k specifies the frame gap between I_t and I_{t+k} and does not determine the robot action frequency. The downstream policy instead learns the action timescale from labeled demonstrations.

IV. EXPERIMENTS

We evaluate TRAIL through qualitative analysis of the representations (Sec. IV-A), simulated control (Secs. IV-B, IV-C), real-robot manipulation (Sec. IV-D), and ablation analysis (Sec. IV-E).

Our experiments are designed to answer the following questions: **Q1:** Can LARM effectively learn dynamics from human-only videos and generalize to out-of-domain robotic embodiments? **Q2:** Does the intent-aligned latent residual improve downstream policy learning compared to static representations and unconstrained predictive models? **Q3:** How does LARM perform across diverse policy architectures?

A. Analysis of pre-trained representations

Pre-training and representation analysis

We first investigate whether LARM validate whether TRAIL successfully learns to extract intent-aligned dynamics during the human-video task-relevant dynamics during pre-training phase.

Pre-training details. We pre-train LARM strictly TRAIL on human-centric videos using the Kinetics-400 training split [36], [41], without any in-domain robotic manipulation

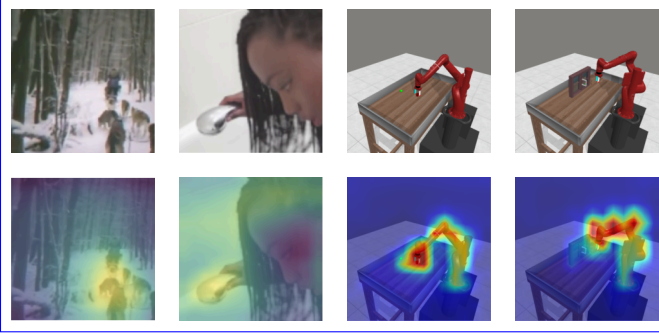


Fig. 4: **Qualitative language grounding after human-video pre-training.** We visualize instruction-conditioned cross-attention maps from TRAIL. Warmer regions indicate larger attention weights and concentrate on task-relevant objects without bounding-box supervision, providing a qualitative check for the language-conditioned prediction objective.

data. To construct the dataset for our prediction-based self-supervised learning, we sample pairs of frames from each video. Specifically, we select the 1st frame as the initial observation and the 96th frame as the target future state. This specific temporal gap ($k = 96$) is chosen to capture meaningful, long-horizon state transitions rather than trivial, high-frequency motions, as in [13]. Kinetics-400 is stored at 25–30 fps, so $k = 96$ spans roughly 3–4 seconds of human activity. The gap is large enough so that the transition is not dominated by encoding noise. k is a hyper-parameter of pre-training on web video and *not* a control horizon, i.e. at deployment the predictor is executed at every control step and its output is consumed as a direction, with magnitude and time scale by the action decoder (Sec. III-B). We adopt DINOv3 (ViT-Base/16) [38] as our foundational visual backbone, and DistilBERT-Base-Uncased [39] as the text backbone. To provide stable and high-quality latent targets without pixel-level reconstruction, as depicted shown in Fig. 2, we employ self-distillation on our visual backbone and formalize a frozen, pre-trained DINOv3 model as a *Teacher* model [1], [42]. Simultaneously, our trainable LARM modules process the initial frame with the language instruction to predict this latent future and optimize the loss $\mathcal{L}_{\text{total}}$ defined in Eq. 6

As shown in Fig. 4, the cross-attention maps concentrate on object regions associated with the corresponding instructions rather than being distributed uniformly over the scene background. This suggests that TRAIL learns to use language to select visual information relevant to future-state prediction, despite receiving no object-level supervision during pre-training. We next evaluate the learned representations on simulated and real-robot manipulation tasks.

B. Simulation results on Franka Kitchen

We validate whether dynamics learned from human videos supports long-horizon robot manipulation across five visuomotor tasks with a simulated 9-DoF Franka Emika

arm [16].

Setup. As shown in Table I, we compare against three categories of baselines: standard visual representation models, vision-only predictive models, and vision-language predictive models. All methods use the same downstream behavioral cloning (BC) architecture with frozen visual backbones, so the policy-learning protocol is held fixed while the representation and its pre-training data differ. We report average success rates over 25 demonstrations across the five tasks, two viewpoints, and three random seeds. Table II separately summarizes the robot-data and annotation requirements of the closest predictive baselines.

Sampled model attention maps in video prediction.

The language description of top and bottom rows are: “A person is sled dog racing” and “A person is washing hair”. Warmer colors (yellow) indicate higher attention weights, while cooler colors (blue) denote lower attention.

In Fig. 4, we validate the visual-language grounding capability of LARM by visualizing the cross-attention maps generated by our bidirectional cross-attention given language instructions. The warmer regions (yellow) demonstrate that LARM localizes the task-related moving objects without any bounding-box supervision. This confirms that the model successfully grounds the language intent into the visual space and filters out task-irrelevant background distractors before prediction.

C. Simulation results

We validate our model on five long-horizon visuomotor tasks in Franka Kitchen [16], a widely adopted continuous control benchmark using a simulated 9-DoF Franka Emika arm.

Setup. As shown in Table I, we compare our method against three major categories of baselines: standard visual representation models, vision-only predictive models, and vision-language predictive models. To ensure a fair comparison, all downstream policies employ the same standard behavioral cloning (BC) architecture, utilizing the respective frozen backbones.

LARM achieves an 81.2% TRAIL achieves the highest average success rate, which significantly outperforms standard visual backbones, surpassing MVP on Franka Kitchen as shown in Table I. It improves over strong frozen-representation baselines such as MVP [14] (71.7%) and Ego4D-trained R3M [2] (69.5%) by +9.5 % and +11.7 %percentage points, respectively. Despite being pre-trained exclusively on human videos without robot embodiment data, LARM successfully generalizes to Even without robot-video pre-training data, the intent stream learned from human videos transfers to an out-of-domain robotic control, providing a strong affirmative answer to Q1.

Compared to arm and produces a representation useful for closed-loop control. Compared with recent vision-language predictive models like such as LaVA-Man [10] (79.2%) and MPI [11] (78.9%), LARM outperforms on TRAIL is competitive on the average and is strongest on several complex articulation tasks. For instance, it achieves

TABLE I: **Results on Franka Kitchen [16].** ~~To ensure a fair comparison~~ We report average success rates over 25 demonstrations across five tasks, two viewpoints, and highlight three random seeds. All rows use the effectiveness of our architectural design same frozen backbone behavioral-cloning downstream and evaluation protocol, we use while the most common backbone and pre-training dataset (Kinetics-300K) and columns expose the upstream data/model scale (ViT-Base) across all downstream evaluations differences. We categorize the baselines Baselines are grouped by their fundamental representation paradigm paradigm; success rates are reported in percent and the best result in each task column is bolded.

Method	Backbone	Pre-training dataset	Turn knob	Open door	Flip switch	Open microwave	Slide door	Average
<i>Visual Representation Models</i>								
DINOv3 [38]	ViT-Base	Internet scale	79.3	43.7	92.3	40.3	100.0	71.3
R3M [2]	ResNet50	Ego4D [43]	53.3	50.7	86.3	59.3	97.0	69.5
MVP [14]	ViT-Base	Multi-dataset	79.0	48.0	90.7	41.0	100.0	71.7
<i>Vision-only Predictive Models</i>								
SiamMAE [9]	ViT-Base	Kinetics [36]	16.8	68.0	36.5	13.5	68.0	40.6
V-JEPA [33]	ViT-Base	Kinetics	21.3	28.0	41.3	12.7	78.7	36.4
ToBo [13]	ViT-Base	Kinetics	57.0	51.0	82.0	55.0	95.0	68.0
<i>Vision-Language Predictive Models</i>								
Voltron [12]	ViT-Base	Something-V2 [44]	76.0	45.3	91.0	41.0	99.3	70.5
MPI [11]	ViT-Base	Ego4D Hand-Obj [43]	89.0	57.7	93.7	54.0	100.0	78.9
LaVA-Man [10]	ViT-Base	OOPP [10], DROID [45], Bridge [46]	90.0	50.7	94.0	61.3	100.0	79.2
Ours	ViT-Base	Internet scale, Kinetics	91.7	56.7	95.7	62.0	100.0	81.2

TABLE II: **Comparison Pre-training data and supervision for the predictive baselines.** Robot Data indicates whether the representation learner uses robot-domain videos or trajectories during pre-training, and Future Cond. indicates whether future-state prediction is part of configurations the representation objective.

Model	Robot Data	Supervision	Future Cond.
MPI	✓	Bbox, image	×
LaVA-MAN	✓	Self-supervised	✓
Ours	×	Self-supervised	✓

62.0% example, TRAIL reaches 62.0% on “Open microwave” and **95.7%** 95.7% on “Flip switch,” outperforming all predictive baselines on these tasks. These results show that our approach to explicitly modeling task-relevant transition directions can be effective. This confirms that explicitly isolating intent-aligned future transitions simplifies downstream policy optimization.

Table II compares summarizes the pre-training configurations of data and supervision used by the predictive models that show similar performance on Franka Kitchen. While, MPI and LaVA-Man rely heavily on in-domain robot data, with MPI further requiring expensive use robot-domain pre-training data, and MPI additionally requires fine-grained annotations (e.g., bounding boxes), our method operates without robot data, using supervision such as bounding boxes. TRAIL uses human-centric internet videos for self-supervised learning on internet-scale human videos. Despite this strictly scalable setup, LARM systematically outperforms robot data-dependent baselines. By sharing the future-conditioning paradigm with LaVA-Man but removing the reliance on in-domain data, we demonstrate pre-training. Under this more scalable pre-training setting, TRAIL matches or outperforms existing predictive models. This supports our central claim that constrained future prediction is effective for extracting task-relevant can extract

robot-relevant physical dynamics from human behavior videos.

C. Multi-task generalization on Meta-World

Franka Kitchen evaluates five tasks that share a single kitchen scene and a common arm configuration. To test whether the task-relevant residual also helps when a single policy must cover heterogeneous scenes, objects and motion primitives, we additionally evaluate on Meta-World MT10 [17], a ten-task multi-task benchmark on a simulated Sawyer arm.

Setup. We collect 100 expert demonstrations per task with the benchmark’s scripted policies and train a single language-conditioned multi-task policy by offline behavioral cloning, with no reinforcement learning and no online interaction. As in Sec. IV-B, every backbone is kept frozen and only the action decoder is trained. All methods, including ours, use the identical pooled action decoder, so the only quantity that differs across rows of Table III is the representation supplied to it. We report mean closed-loop success over three random seeds, evaluating 50 episodes per task and seed. All methods use the same demonstration splits, random seeds, action decoder, camera and proprioceptive inputs, and evaluation configurations. The evaluation seeds are disjoint from all demonstration-collection seeds, preventing policies from being scored on object configurations seen during data collection.

Visual input. We render the corner2 third-person camera at 224×224 . The policy receives a single RGB frame together with a 4-dimensional proprioceptive state (end-effector position and gripper aperture), and no wrist camera, no depth channel and no frame stacking are used. Object positions are randomized by the benchmark at every reset, and the evaluation seeds control that randomization.

TRAIL reaches a 78.2% MT10 average, ahead of the strongest baseline R3M [2] (70.0%). The aggregate margin

TABLE III: **Evaluation of real-world multi-task manipulation Results on Meta-World MT10 [17].** A language-conditioned policy is trained per method by offline behavioral cloning on 100 expert demonstrations per task with a frozen visual backbone and an identical action decoder. We compare the report mean success rates of our intent-aligned model against CLIPort over three random seeds, MPI, with 50 closed-loop episodes per task and LaVA-Man across seven language-conditioned tasks. The right panel details our robot hardware setup. All methods use the same demonstration splits, random seeds, camera and proprioceptive inputs, action decoder, and evaluation configurations; evaluation seeds are disjoint from demonstration-collection seeds. Methods are grouped by the diverse set representation paradigms of experimental objects Table I. Our approach outperforms Task names abbreviate the baselines in average success rate, exhibiting particularly strong zero-shot generalization on unseen tasks. MT10 suite: Drawer-O/C denote drawer open/close and Window-O/C denote window open/close.

Method	Reach	Push	Pick-Place	Door	Drawer-O	Drawer-C	Button	Peg-Insert	Window-O	Window-C	Average
<i>Visual Representation Models</i>											
DINOv3 [38]	36.0	38.0	18.0	100.0	92.0	100.0	96.0	28.0	80.0	100.0	68.8
R3M [2]	50.0	32.0	2.0	100.0	100.0	100.0	100.0	26.0	90.0	100.0	70.0
VC-1 [19]	32.0	2.0	0.0	80.0	86.0	36.0	28.0	2.0	22.0	100.0	38.8
<i>Vision-only Predictive Models</i>											
ToBo [13]	28.0	38.0	2.0	100.0	100.0	100.0	100.0	34.0	96.0	100.0	69.8
<i>Vision-Language Predictive Models</i>											
MPI [11]	36.0	24.0	4.0	80.0	100.0	100.0	100.0	12.0	72.0	100.0	62.8
Ours	44.0	70.0	40.0	100.0	100.0	100.0	100.0	33.0	95.0	100.0	78.2

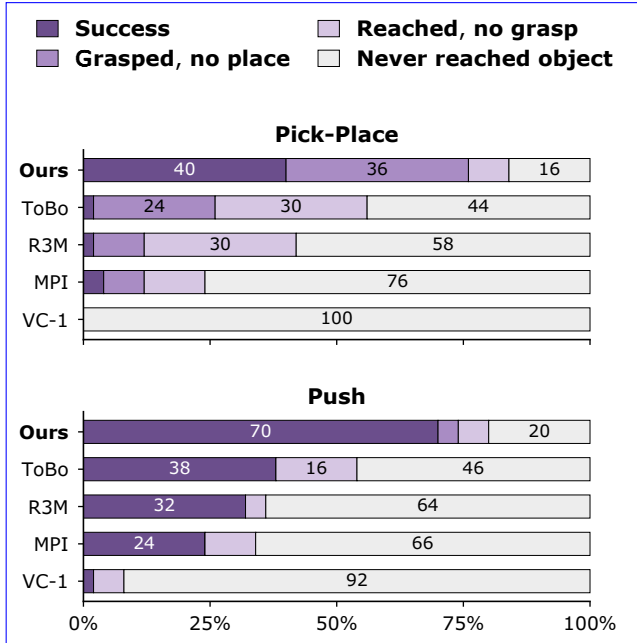


Fig. 5: **Real-robot experiment setup Failure stages on Meta-World Push and Pick-Place.** Episodes are grouped by the furthest stage reached. We used various object (left) and a UR5 robot arm with a Robotiq-2F-85 gripper for tabletop manipulation tasks. Push has no grasp stage.

is modest, but it is not spread evenly across the suite, and the per-task pattern is the informative part. Six of the ten tasks — Door, both Drawer tasks, Button and Window-Close — are at or near saturation for every method that yields a usable representation, and therefore carry no discriminative signal. The entire gap comes from two tasks: “Pick-Place” (40.0% vs. at most 18.0% for any baseline) and “Push” (70.0% vs. at most 38.0%). Fig. 5 further decomposes Push

and Pick-Place outcomes by the furthest stage reached. TRAIL reduces failures before object contact and shifts the remaining Pick-Place failures toward the later placement stage.

These two tasks share a property that the saturated ones lack: the target object moves while the gripper approaches it, so the correct action depends on where the scene is going rather than only on where it currently is. A static embedding is sufficient to replay a memorized opening trajectory, but it under-determines the action once the goal is non-stationary. Conditioning on the task-relevant residual Δ_z supplies exactly the missing directional signal, which is why the benefit is invisible on the articulation tasks and decisive on the two dynamic ones. Sec. IV-E isolates this effect by removing Δ_z from our own policy rather than comparing across representations.

The baseline ordering also differs from Franka Kitchen in an informative way. VC-1 [19] (38.8%), a masked-autoencoding model, transfers markedly worse than the contrastively pre-trained R3M despite a larger backbone. Under our frozen-backbone, shallow-decoder protocol, this result suggests that reconstruction-oriented features are less linearly separable for these control tasks.

D. Real-robot experiment

We validate LARM in the real-robot environment using seven tabletop manipulation tasks further validate TRAIL on a physical UR5 setup where language-conditioned control must handle object appearance shifts, clutter, and contact-rich motions.

Setup. The hardware setup includes a UR5 robotic arm with a Robotiq-2F-85 gripper and an Intel RealSense RGB-D camera capturing wrist-mounted Intel RealSense camera observing a 60 cm $\times$ 30 cm workspace (Fig. 7). To adapt our model to real-world control, we The policy uses RGB images

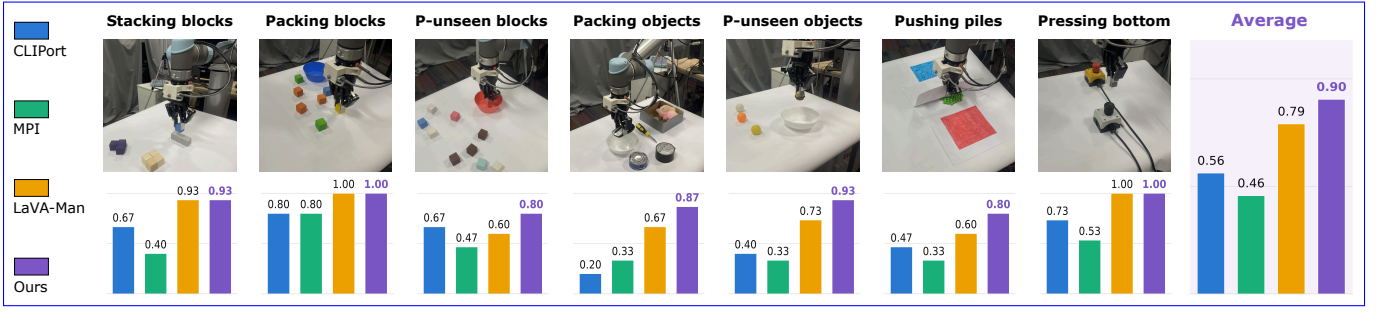


Fig. 6: **Evaluation of real-robot manipulation.** We compare our method with CLIPort, MPI, and LaVA-Man across seven language-conditioned tabletop tasks. Task names abbreviation: P-unseen blocks denote Packing unseen blocks and P-unseen objects denote Packing unseen objects.

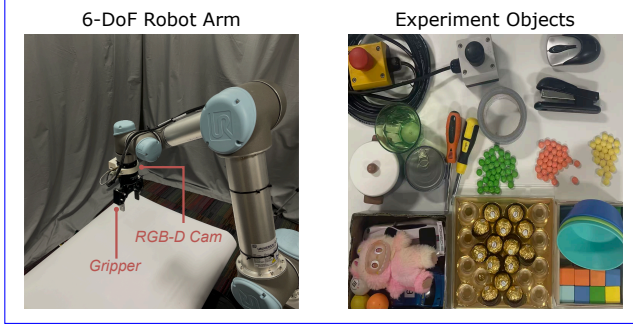


Fig. 7: **Real-robot setup.** A wrist-mounted camera provides RGB observations for UR5. Depth is not used.

only, and depth measurements are not used. We collect 200 language-annotated expert demonstrations. We use this dataset to jointly fine-tune our method and three baselines (CLIPort and train each method under the same multi-task protocol, comparing TRAIL with CLIPort [20], MPI [11], MPI, and LaVA-Man) into a single unified multi-task policy, requiring the models to condition their actions on language instructions [10] using identical demonstrations and task instructions. Each trained policy must select actions conditioned on the language command rather than on a task ID alone. To test zero-shot generalization, five colored blocks and five novel objects are strictly excluded from the training data. We evaluate the co-trained policy across and introduced only at evaluation time. In addition, pushing piles and pressing buttons are completely unseen tasks, with no demonstrations of either task in the training data. We evaluate seven subtasks: stacking blocks, packing blocks/objects, packing unseen blocks/objects, pushing piles, and pressing buttons. For each subtask, we conduct 15 independent evaluation trials.

As shown in Fig. 6, our method successfully executes the tasks and TRAIL consistently outperforms the baselines. On average, our model in real-world multi-task manipulation. It achieves an overall success rate of over 85%, significantly surpassing 90%, substantially higher than the strongest baseline, LaVA-Man (~7080%), as well as and well above CLIPort and MPI (~5055%). In Fig. 8, we demonstrate example pick-and-place affordance maps. Our method exhibits strong zero-shot generalization, achieving

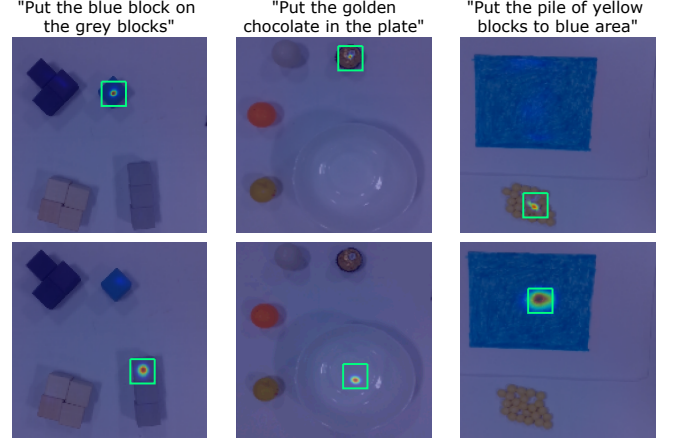


Fig. 8: **Sampled affordance maps.** We obtain the visualize TRAIL’s predicted affordance maps of the LARM model at the start and end stages in-of real-robot tabletop manipulation tasks. The responses follow the commanded object and target regions, providing qualitative evidence for language-conditioned spatial grounding.

a 100% success rate The strongest evidence appears in the held-out settings: TRAIL reaches 93% success on “Packing unseen objects” (vs.80 73% for LaVA-Man) and an 80% success rate on “Packing unseen blocks” (a 20% 20-point absolute improvement over LaVA-Man). Furthermore, for tasks with complex contact dynamics like In the contact-rich “Pushing piles,” our method reaches an 80% success rate, doubling, surpassing the performance of MPI and LaVA-Man. These results confirm that our intent-aligned pre-training effectively grounds language instructions to visual features, enabling robust continuous control without overfitting Together with the affordance visualizations in Fig. 8, these results support the claim that learning task-relevant latent changes improves robust real-world language-conditioned control and reduces overfitting to the training objects.

E. Ablation study

A core contribution of LARM is challenging the standard representation learning paradigm, which typically discards the predictive decoder TRAIL keeps the intent stream instead

TABLE IV: Ablation study on latent residuals. We evaluating the impact of retaining the predictive module to compute Δ_z on the Franka Kitchen benchmark reports per-task and average success rates; Meta-World MT10 reports the average under the same frozen-backbone, decoder protocol as Table I [16].

Variant	Input	Franka Kitchen					MT10 Avg.
		Knob	Door	Switch	Micro.	Slide	
Encoder-Only	z_t	79.3	46.0	91.0	56.3	100.0	74.5
Predictor-Only	$\hat{z}_{t+k}^{\text{int}}$	86.3	46.3	89.7	55.3	100.0	75.5
Absolute pair	$z_t, \hat{z}_{t+k}^{\text{int}}$	92.0	45.0	88.7	55.0	100.0	76.1
Full	z_t, Δ_z	91.7	56.7	95.7	62.0	100.0	81.2

of discarding the predictive head after pre-training [2], [8]. We validate the impact of explicitly retaining the intent-aligned predictor by conducting an ablation study on Franka Kitchen [16].

We compare three variants isolate this design choice by comparing policy inputs that keep either the current state, the predicted future state, the absolute pair, or the residual direction. We compare four variants of TRAIL based on the inputs provided to the downstream policy: **LARM (Encoder-Only)**: The downstream policy is conditioned only on static current visual features *Encoder-Only*, using only z_t . **LARM (Predictor-Only)**: The policy is conditioned exclusively on the predicted future latent state $\hat{z}_{t+1}$, *Predictor-Only*, using $\hat{z}_{t+k}^{\text{int}}$ generated by the predictor. **LARM (Full)**: The policy is conditioned on both the current state and the explicitly computed intent-aware latent residual: *Absolute pair*, using both z_t and $\hat{z}_{t+k}^{\text{int}}$, and the *Full model* $\pi(z_t, \Delta_z)$.

As shown in Table IV, while the shows that future information is useful, but the residual form is the most effective way to expose it to the policy. On Franka Kitchen, *Encoder-Only* variant achieves a competitive average success rate of reaches 74.5%, it struggles with complex, long-horizon articulation tasks such as average success but remains weak on “Open door” (46.0%) and “Open microwave” (56.3%). Interestingly, the *Predictor-Only* variant provides only a marginal improvement. Supplying only the predicted future state improves little on Franka (75.5% average), indicating that simply supplying the policy with a raw predicted future state is insufficient for precise motor control. By explicitly computing the actionable latent residual Δ_z , the *Full* LARM model achieves a dramatic +6.7% absolute improvement over the baseline, reaching an), and giving the absolute pair ($z_t, \hat{z}_{t+k}^{\text{int}}$) is still below the full residual representation (76.1% vs. 81.2% average success rate. This confirms that providing a direct, intent-aligned directional vector effectively simplifies the downstream policy learning objective compared to implicitly forcing the network to infer the difference between z_t and $\hat{z}_{t+1}$ (%). The same pattern appears in the MT10 ablation column, where the full model obtains the best average among the four variants. These results show that the gain is not merely from adding a future feature, but from making the

task-relevant transition Δ_z explicit so that the downstream decoder receives both the current state and a direction of task progress.

V. CONCLUSION

We proposed **LARM**, a prediction-based **TRAIL**, a self-supervised representation learning framework learning framework that learns predictive representations for robot manipulation policies. The key idea is to estimate intent-aware latent residuals that encourage the model to focus on task-relevant changes, task-relevant latent residuals and then use these latent residuals for effective policy learning. Simulation and real-robot experiments demonstrate that our model benefits the learning for both continuous control and tabletop arrangement policies, outperforming prior methods. In the future, we **Future work** will extend our framework to more diverse manipulation tasks and real-world robotic settings.

REFERENCES

- [1] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging properties in self-supervised vision transformers,” in *Int. Conf. Comput. Vis.*, 2021.
- [2] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, “R3M: A universal visual representation for robot manipulation,” in *Conf. Robot Learn.*, 2023.
- [3] Y. J. Ma, S. Sodhani, D. Jayaraman, O. Bastani, V. Kumar, and A. Zhang, “VIP: Towards universal visual reward and representation via value-implicit pre-training,” in *Int. Conf. Learn. Represent.*, 2023.
- [4] P. Hewage, A. Behera, M. Trovati, E. Pereira, M. Ghahremani, F. Palmieri, and Y. Liu, “Temporal convolutional neural (tcn) network for an effective weather forecasting using time-series data from the local weather station,” *Soft Computing*, vol. 24, no. 21, pp. 16453–16482, 2020.
- [5] K. Zakka, A. Zeng, P. Florence, J. Tompson, J. Bohg, and D. Dwibedi, “Xirl: Cross-embodiment inverse reinforcement learning,” in *Conf. Robot Learn.*, 2022.
- [6] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, “Dream to control: Learning behaviors by latent imagination,” *arXiv preprint arXiv:1912.01603*, 2019.
- [7] C. Finn and S. Levine, “Deep visual foresight for planning robot motion,” in *IEEE Int. Conf. Robot. Autom.*, 2017.
- [8] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” in *IEEE/CVF Conf. Comput. Vis. Pattern Recog.*, 2022.
- [9] A. Gupta, J. Wu, J. Deng, and F.-F. Li, “Siamese masked autoencoders,” in *Adv. Neural Inform. Process. Syst.*, 2023.
- [10] C. Zhu, H. Wang, Y. L. Pang, and C. Oh, “LaVA-Man: Learning visual action representations for robot manipulation,” in *Conf. Robot Learn.*, 2025.
- [11] J. Zeng, Q. Bu, B. Wang, W. Xia, L. Chen, H. Dong, H. Song, D. Wang, D. Hu, P. Luo *et al.*, “Learning manipulation by predicting interaction,” *arXiv preprint arXiv:2406.00439*, 2024.
- [12] S. Karamcheti, S. Nair, A. S. Chen, T. Kollar, C. Finn, D. Sadigh, and P. Liang, “Language-driven representation learning for robotics,” in *Robotics: Science and Systems*, 2023.
- [13] T. Kim, D. Han, B. Heo, J. Park, and S. Yun, “Token Bottleneck: One token to remember dynamics,” in *Adv. Neural Inform. Process. Syst.*, 2025.
- [14] I. Radosavovic, T. Xiao, S. James, P. Abbeel, J. Malik, and T. Darrell, “Real-world robot learning with masked visual pre-training,” in *Conf. Robot Learn.*, 2023.
- [15] T. Xiao, I. Radosavovic, T. Darrell, and J. Malik, “Masked visual pre-training for motor control,” *arXiv preprint arXiv:2203.06173*, 2022.
- [16] A. Gupta, V. Kumar, C. Lynch, S. Levine, and K. Hausman, “Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning,” in *Conf. Robot Learn.*, 2019.
- [17] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, “Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning,” in *Conf. Robot Learn.*, 2020.

- [18] Y. J. Ma, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman, "Liv: Language-image representations and rewards for robotic control," in *Int. Conf. Mach. Learn.*, 2023.
- [19] A. Majumdar, K. Yadav, S. Arnaud, J. Ma, C. Chen, S. Silwal, A. Jain, V.-P. Berges, T. Wu, J. Vakil *et al.*, "Where are we in the search for an artificial visual cortex for embodied intelligence?" *Adv. Neural Inform. Process. Syst.*, 2023.
- [20] M. Shridhar, L. Manuelli, and D. Fox, "Cliport: What and where pathways for robotic manipulation," in *Conf. Robot Learn.*, 2022.
- [21] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu *et al.*, "Rt-1: Robotics transformer for real-world control at scale," *arXiv preprint arXiv:2212.06817*, 2022.
- [22] N. Tishby, F. C. Pereira, and W. Bialek, "The information bottleneck method," *arXiv preprint physics/0004057*, 2000.
- [23] B. You and H. Liu, "Multimodal information bottleneck for deep reinforcement learning with multiple sensors," *Neural Networks*, vol. 176, p. 106347, 2024.
- [24] S. Bai, W. Zhou, P. Ding, W. Zhao, D. Wang, and B. Chen, "Rethinking latent redundancy in behavior cloning: An information bottleneck approach for robot manipulation," in *Int. Conf. Mach. Learn.*, 2025.
- [25] S. Dasari, F. Ebert, S. Tian, S. Nair, B. Bucher, K. Schmeckpeper, S. Singh, S. Levine, and C. Finn, "RoboNet: Large-scale multi-robot learning," in *Conf. Robot Learn.*, 2020.
- [26] Y. Du, S. Yang, B. Dai, H. Dai, O. Nachum, J. Tenenbaum, D. Schuurmans, and P. Abbeel, "Learning universal policies via text-guided video generation," in *Adv. Neural Inform. Process. Syst.*, 2023.
- [27] K. Black, M. Nakamoto, P. Atreya, H. R. Walke, C. Finn, A. Kumar, and S. Levine, "Zero-Shot robotic manipulation with pre-trained image-editing diffusion models," in *Int. Conf. Learn. Represent.*, 2024.
- [28] D. Schmidt and M. Jiang, "Learning to act without actions," in *Int. Conf. Learn. Represent.*, 2024.
- [29] J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps *et al.*, "Genie: Generative interactive environments," in *Int. Conf. Mach. Learn.*, 2024.
- [30] S. Ye, J. Jang, B. Jeon, S. Joo, J. Yang, B. Peng, A. Mandlekar, R. Tan, Y.-W. Chao, B. Y. Lin *et al.*, "Latent action pretraining from videos," in *Int. Conf. Learn. Represent.*, 2025.
- [31] AgiBot-World-Contributors, Q. Bu, J. Cai, L. Chen, X. Cui, Y. Ding, S. Feng, S. Gao, X. He, X. Hu, X. Huang *et al.*, "AgiBot World Colosseum: A large-scale manipulation platform for scalable and intelligent embodied systems," *arXiv preprint arXiv:2503.06669*, 2025.
- [32] Y. LeCun, "A path towards autonomous machine intelligence version 0.9.2, 2022-06-27," *Open Review*, vol. 62, 2022.
- [33] A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas, "Revisiting feature prediction for learning visual representations from video," *Trans. Mach. Learn. Res.*, 2024.
- [34] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, "Self-supervised learning from images with a joint-embedding predictive architecture," in *IEEE/CVF Conf. Comput. Vis. Pattern Recog.*, 2023.
- [35] P. Benchetrit *et al.*, "MC-JEPA: A joint-embedding predictive architecture for self-supervised learning of motion and content features," *arXiv preprint arXiv:2307.12698*, 2023.
- [36] W. Kay, J. Carreira, K. Simonyan, B. Zhang, C. Hillier, S. Vijayanarasimhan, F. Viola, T. Green, T. Back, P. Natsev *et al.*, "The kinetics human action video dataset," *arXiv preprint arXiv:1705.06950*, 2017.
- [37] L. H. Li, P. Zhang, H. Zhang, J. Yang, C. Li, Y. Zhong, L. Wang, L. Yuan, L. Zhang, J.-N. Hwang *et al.*, "Grounded language-image pre-training," in *IEEE/CVF Conf. Comput. Vis. Pattern Recog.*, 2022.
- [38] O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, C. Jose, V. Khalidov, M. Szafraniec, S. Yi, M. Ramamonjisoa *et al.*, "Dinov3," *arXiv preprint arXiv:2508.10104*, 2025.
- [39] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.
- [40] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zhohal *et al.*, "V-jepa 2: Self-supervised video models enable understanding, prediction and planning," *arXiv preprint arXiv:2506.09985*, 2025.
- [41] J. Carreira and A. Zisserman, "Quo vadis, action recognition? a new model and the kinetics dataset," in *IEEE/CVF Conf. Comput. Vis. Pattern Recog.*, 2017.
- [42] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. Richemond, E. Buchatskaya, C. Doersch, B. Avila Pires, Z. Guo, M. Gheshlaghi Azar *et al.*, "Bootstrap your own latent: A new approach to self-supervised learning," in *Adv. Neural Inform. Process. Syst.*, 2020.
- [43] K. Grauman, A. Westbury, E. Byrne, Z. Chavis, A. Furnari, R. Girdhar, J. Hamburger, H. Jiang, M. Liu, X. Liu *et al.*, "Ego4d: Around the world in 3,000 hours of egocentric video," in *IEEE/CVF Conf. Comput. Vis. Pattern Recog.*, 2022.
- [44] R. Goyal, S. Ebrahimi Kahou, V. Michalski, J. Materzynska, S. Westphal, H. Kim, V. Haenel, I. Fruend, P. Yianilos, M. Mueller-Freitag *et al.*, "The 'something something' video database for learning and evaluating visual common sense," in *Int. Conf. Comput. Vis.*, 2017.
- [45] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis *et al.*, "DROID: A large-scale in-the-wild robot manipulation dataset," in *RSS Workshop: Data Generation for Robotics*, 2024.
- [46] H. R. Walke, K. Black, T. Z. Zhao, Q. Vuong, C. Zheng, P. Hansen-Estruch, A. W. He, V. Myers, M. J. Kim, M. Du *et al.*, "Bridgedata v2: A dataset for robot learning at scale," in *Conf. Robot Learn.*, 2023.